ECE 36800
PA 2: Collision Detection

Due Monday, July 20th at 11:59 PM

Goal.

Your goal in this project is to implement an efficient collision detector in 2D space. Your programmer will load a set of "obstacles" T, and then respond to classification queries one at a time. Each query will consist of a set of coordinates for an object plus a radius q, and your program should print the number of points in T within q. You will be graded both on accuracy and whether your program executes in a reasonable amount of time.

Input & Output

Your program will be run as `./pa2 <input>` where `<input>` is the name of a text file containing the obstacles. The input file contains two space separated integers in the format `<x> <y>`, with each obstacle separated by a newline. `<x>` and `<y>` may be any valid signed 32 bit integer (this notably includes negative numbers).

Your program will then read queries from standard input containing three space separated integers in the format `<x> <y> <radius>`, with each query separated by a newline. For each query, print out the number of collisions within the query circle, followed by a newline.

The program will terminate upon reaching the end of input, which can be triggered using `Ctrl + D`. Upon termination, you should free up all allocated memory then exit gracefully with `EXIT_SUCCESS`.

You are not required to validate the format of the input file or handle invalid commands. If you receive an invalid file or an invalid query, you may handle this as you see fit.

Example

Consider the following set of points:

```
3 5
5 1
0 6
```

Given a query point $(0, 5)$ with a radius of 1, our program should output 1 as $(0, 6)$ collides. The point $(0, 5)$ with a radius of 3 would output 2 as it contains both the points $(0, 5)$ and $(3, 5)$. The program standard input and output for this example would look like:

```
0 5 1
1
0 5 3
2
```

Grading

This programming assignment will be graded based on both the correctness and memory safety of your implementation. The breakdown is as follows:

- Each test case will be worth the same amount of points, and will be all-or-nothing (i.e. you either have the correct output or you don't).
- A 50% penalty will be applied for any test cases on which your submission has a memory error or a memory leak.
- You will receive a 0 if your submission fails to terminate within a 10-minute time limit.
- Individual test cases will have a reasonable timeout based on the size of the problem, to ensure you are using the appropiate data structures (i.e. trees) to solve this problem. If you fail this timeout, you will receive a 0 on the test case. The autograder will tell you if you are failing any cases due to timeout.
- You will only be graded based on the output to `stdout`. Anything printed to `stderr` will be ignored. You may use this for debugging, error messages, or program prompts if you wish.

You may submit to the Gradescope autograder as often as you'd like before the deadline. Only your active submission (by default the most recent) will be counted. While the score given to you by the autograder is likely a good indicator of your final grade on the assignment, we reserve the right to add additional test cases after the submission deadline.

Submission Instructions.

Submit any source/header files with your implementation, as well as a Makefile that builds a target called `pa2`, to Gradescope. Note that to receive points, your submission must work on `eceprog`. (See the syllabus for more details.)
If you choose to do the optional writeup, submit it to the assignment on Brightspace. The writeup can be formatted however you want, but here's a guideline of some questions you may want to think about when writing it:

- What data structures did you choose to use, and why? Are there different choices that would have made a more efficient solution?
- Did you implement any extra error handling?
- What other factors did you consider? How did they influence your final implementation?
- What, if anything, did you find most challenging about this project? Was there anything in particular you enjoyed/disliked?
- If you took a particularly creative approach to solving this problem, or ended up implementing something we didn't ask for, tell me about it!